ВикипедиЯ
Свободная энциклопедия

DES

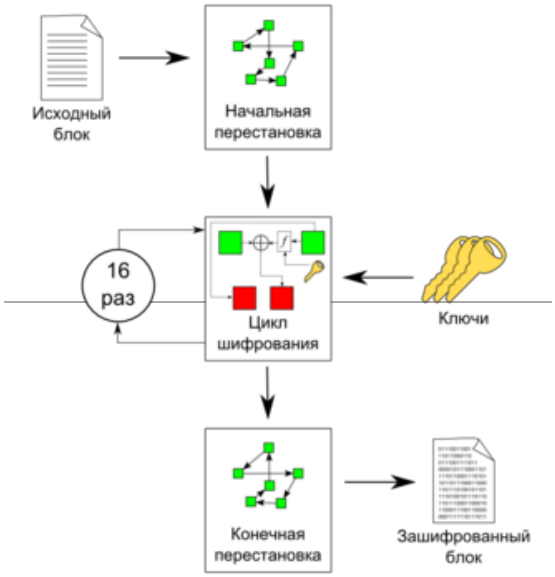
Материал из Википедии — свободной энциклопедии

DES (англ. *Data Encryption Standard*) — алгоритм для симметричного шифрования, разработанный фирмой IBM и утверждённый правительством США в 1977 году как официальный стандарт (FIPS 46-3). Размер блока для DES равен 64 битам. В основе алгоритма лежит сеть Фейстеля с 16 циклами (раундами) и ключом, имеющим длину 56 бит. Алгоритм использует комбинацию нелинейных (S-блоки) и линейных (перестановки E, IP, IP-1) преобразований. Для DES рекомендовано несколько режимов:

- ECB (англ. ***e**lectronic **c**ode **b**ook*) — режим «электронной кодовой книги» (простая замена);
- CBC (англ. ***c**ipher **b**lock **c**haining*) — режим сцепления блоков;
- CFB (англ. ***c**ipher **f**eed **b**ack*) — режим обратной связи по шифротексту;
- OFB (англ. ***o**utput **f**eed **b**ack*) — режим обратной связи по выходу;
- Counter Mode (CM) — режим счётчика.

Прямым развитием DES в настоящее время является алгоритм Triple DES (3DES). В 3DES шифрование/расшифрование выполняются путём троекратного выполнения алгоритма DES.

DES, Data Encryption Standard



Создатель	IBM
Создан	1977
Опубликован	1977 год
Размер ключа	56 бит + 8 проверочных
Размер блока	64 бит
Число раундов	16
Тип	Сеть Фейстеля

Медиафайлы на Викискладе

История

В 1972 году было проведено исследование потребности правительства США в компьютерной безопасности. Американское «национальное бюро стандартов» (НБС) (ныне известное, как NIST — «национальный институт стандартов и технологий») определило необходимость в общеправительственном стандарте шифрования некритичной информации.

НБС проконсультировалось с АНБ (агентством национальной безопасности США) и 15 мая 1973 года объявило первый конкурс на создание шифра. Были сформулированы строгие требования к новому шифру. Фирма IBM представила на конкурсе разработанный ею шифр, называемый «Люцифер» (Lucifer). Шифры ни одного из конкурсантов (включая «Люцифер») не обеспечивали выполнение всех требований. В течение 1973—1974 годов IBM доработала свой «Люцифер»: использовала в его основе алгоритм Хорста Фейстеля, созданный ранее. 27 августа 1974 года начался второй конкурс. На сей раз шифр «Люцифер» сочли приемлемым.

17 марта 1975 года предложенный алгоритм DES был издан в «Федеральном реестре». В 1976 году для обсуждения DES было проведено два открытых симпозиума. На симпозиумах жёсткой критике подверглись изменения, внесённые в алгоритм организацией АНБ. АНБ уменьшило первоначальную длину ключа и S-блоки (блоки подстановки), критерии проектирования которых не раскрывались. АНБ подозревалось в сознательном ослаблении алгоритма с той целью, чтобы АНБ могло легко просматривать зашифрованные сообщения. Сенат США проверил действия АНБ и в 1978 году опубликовал заявление, в котором сообщалось следующее:

- в процессе разработки алгоритма представители АНБ убедили создателей DES в том, что уменьшенной длины ключа более чем достаточно для всех коммерческих приложений;
- представители АНБ косвенно помогали в разработке S-перестановок;
- окончательная версия алгоритма была, по мнению проверяющих, лучшим алгоритмом шифрования, к тому же лишённым статистической или математической слабости;
- представители АНБ никогда не вмешивались в разработку алгоритма DES.

В 1990 году Эли Бихам (Eli Biham) и Ади Шамир (Adi Shamir) провели независимые исследования по дифференциальному криптоанализу — основному методу взлома блочных алгоритмов симметричного шифрования. Эти исследования сняли часть подозрений в скрытой слабости S-перестановок. S-блоки алгоритма DES оказались намного более устойчивыми к атакам, чем если бы их выбрали случайно. Это означает, что такая техника анализа была известна АНБ ещё в 1970-х годах.

Алгоритм DES удалось «взломать» за 39 дней с помощью огромной сети, состоящей из десятков тысяч компьютеров[1].

Общественная организация «EFF», занимающаяся проблемами информационной безопасности и личной тайны в сети Интернет, инициировала исследование «DES Challenge II» с целью выявления проблем DES. В рамках исследования сотрудники фирмы «RSA Laboratory» построили суперкомпьютер стоимостью 250 тыс. долл. В 1998 году суперкомпьютер выполнил расшифровку данных, закодированных методом DES с использованием 56-битного ключа, менее чем за три дня. Суперкомпьютер получил название «EFF DES Cracker». Специально по этому случаю учёные организовали пресс-конференцию и с беспокойством говорили о том, что злоумышленники вряд ли упустят случай воспользоваться подобной уязвимостью.

Некоторые правительственные чиновники и специалисты утверждали, что для взлома кода DES требуется суперкомпьютер стоимостью в несколько миллионов долларов. «Правительству пора признать ненадёжность DES и поддержать создание более мощного стандарта шифрования», — сказал президент EFF Барри Штайнхардт. Экспортные ограничения, накладываемые правительством США, касаются технологий шифрования по ключам длиной

более 40 бит. Однако, как показали результаты эксперимента RSA Laboratory, существует возможность взлома и более мощного кода. Проблема усугублялась тем, что стоимость постройки подобного суперкомпьютера неуклонно снижалась. «Через четыре-пять лет такие компьютеры будут стоять в любой школе», — заявил Джон Гилмор, руководитель проекта «DES Challenge» и один из основателей EFF.

DES является блочным шифром. Чтобы понять, как работает DES, необходимо рассмотреть принцип работы блочного шифра, сеть Фейстеля.

Блочный шифр

Входными данными для блочного шифра служат:

- блок размером n бит;
- ключ размером k бит.

На выходе (после применения шифрующих преобразований) получается зашифрованный блок размером n бит, причём незначительные различия входных данных, как правило, приводят к существенному изменению результата.

Блочные шифры реализуются путём многократного применения к блокам исходного текста некоторых базовых преобразований.

Базовые преобразования:

- сложное преобразование на одной локальной части блока;
- простое преобразование между частями блока.

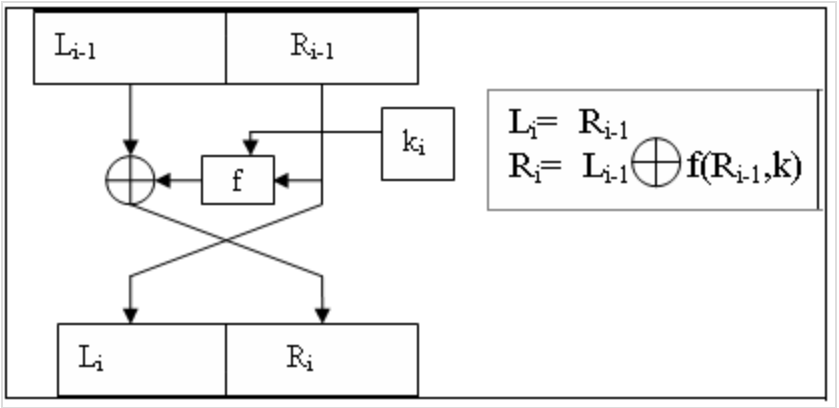


Рис. 1. Прямое преобразование сетью Фейстеля

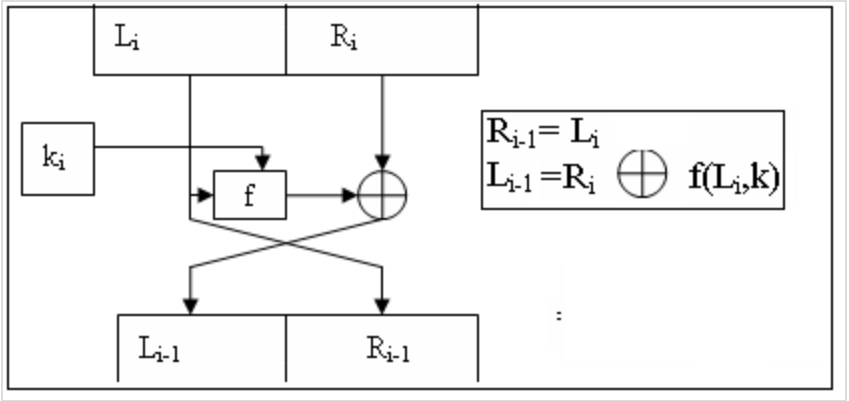


Рис. 2. Обратное преобразование сетью Фейстеля

Так как преобразования производятся поблочно, требуется разделение исходных данных на блоки необходимого размера. При этом формат исходных данных не имеет значения (будь то текстовые документы, изображения или другие файлы). Данные должны интерпретироваться в двоичном виде (как последовательность нулей и единиц) и только после этого должны разбиваться на блоки. Все вышеперечисленное может осуществляться как программными, так и аппаратными средствами.

Преобразования сетью Фейстеля

Это преобразование над векторами (блоками), представляющими собой левую и правую половины регистра сдвига. В алгоритме DES используются прямое преобразование сетью Фейстеля в шифровании (см. Рис.1) и обратное преобразование сетью Фейстеля в расшифровании (см. Рис.2).

Схема шифрования алгоритма DES

Схема шифрования алгоритма DES указана на Рис.3.

Исходный текст — блок 64 бит.

Процесс шифрования состоит из начальной перестановки, 16 циклов шифрования и конечной перестановки.

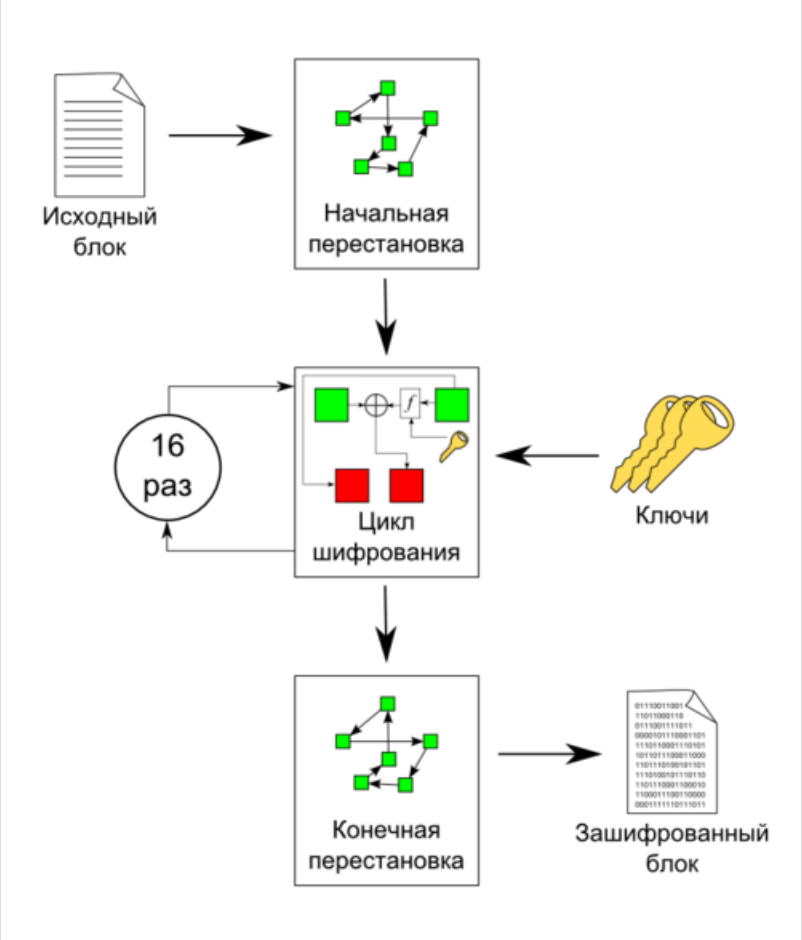


Рис.3 Схема шифрования алгоритма DES

Начальная перестановка

Исходный текст T (блок 64 бит) преобразуется с помощью начальной перестановки IP которая определяется таблицей 1:

Таблица 1. Начальная перестановка IP

58	50	42	34	26	18	10	2	60	52	44	36	28	20	12	4
62	54	46	38	30	22	14	6	64	56	48	40	32	24	16	8
57	49	41	33	25	17	9	1	59	51	43	35	27	19	11	3
61	53	45	37	29	21	13	5	63	55	47	39	31	23	15	7

По таблице первые 3 бита результирующего блока $\mathrm{IP}(T)$ после начальной перестановки IP являются битами 58, 50, 42 входного блока T, а его 3 последние бита являются битами 23, 15, 7 входного блока.

Циклы шифрования

Полученный после начальной перестановки 64-битовый блок IP(T) участвует в 16 циклах преобразования Фейстеля.

— 16 циклов преобразования Фейстеля:

Разбить IP(T) на две части L_0, R_0, где L_0, R_0 — соответственно 32 старших битов и 32 младших битов блока T_0 IP(T)= L_0R_0

Пусть $T_{i-1} = L_{i-1}R_{i-1}$ результат (i-1) итерации, тогда результат i-ой итерации $T_i = L_iR_i$ определяется:

$$L_i = R_{i-1}$$
$$R_i = L_{i-1} \oplus f(R_{i-1}, k_i)$$

Левая половина L_i равна правой половине предыдущего вектора $L_{i-1}R_{i-1}$. А правая половина R_i — это битовое сложение L_{i-1} и $f(R_{i-1}, k_i)$ по модулю 2.

В 16-циклах преобразования Фейстеля функция f играет роль шифрования. Рассмотрим подробно функцию f.

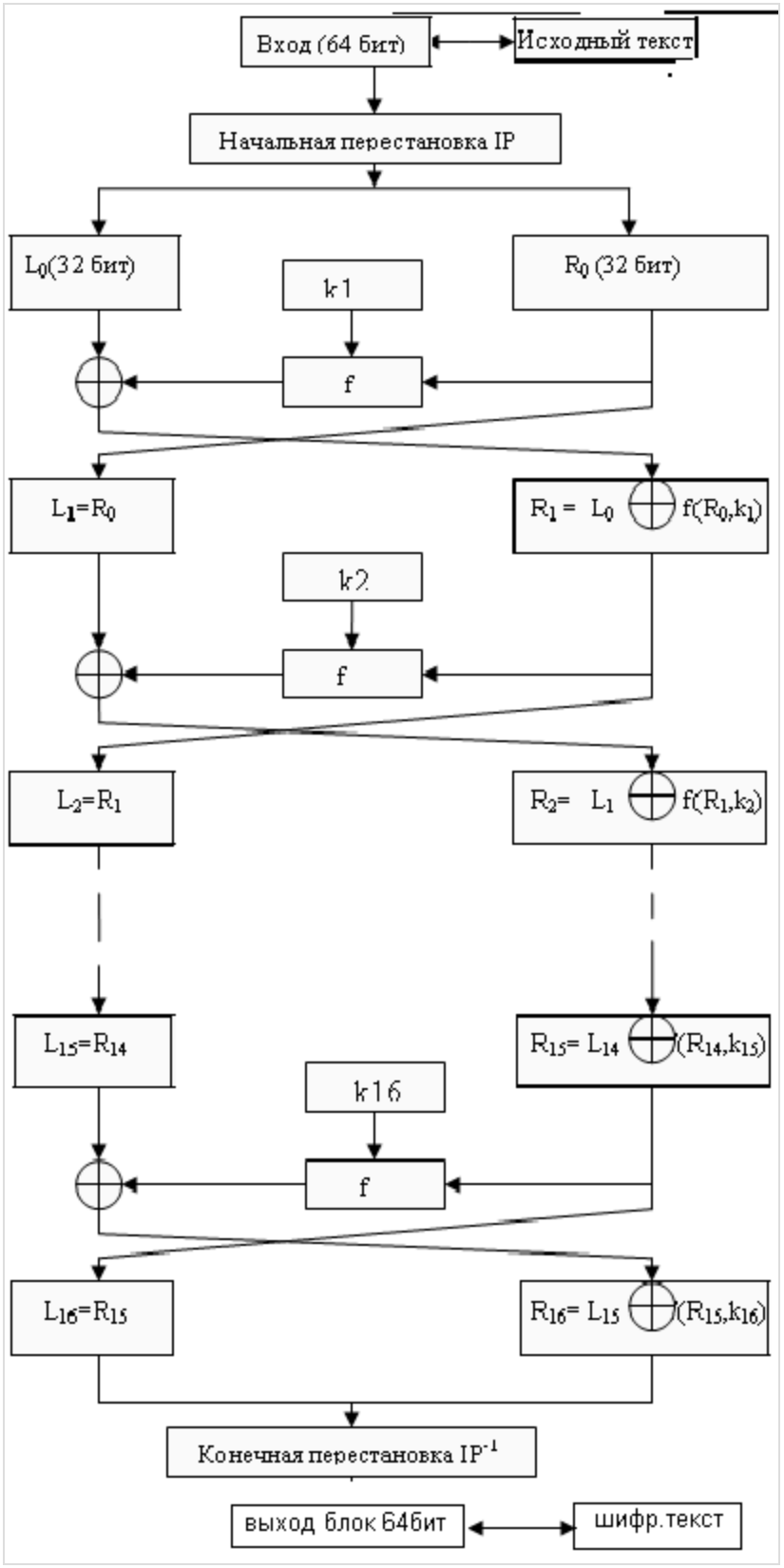


Рис.4 Подробная схема шифрования алгоритма DES

Основная функция шифрования (функция Фейстеля)

Аргументами функции f являются 32-битовый вектор R_{i-1} и 48-битовый ключ k_i, который является результатом преобразования 56-битового исходного ключа шифра k. Для вычисления функции f последовательно используются

1. функция расширения E,
2. сложение по модулю 2 с ключом k_i
3. преобразование S, состоящее из 8 преобразований S-блоков $\mathrm{S}_1, \mathrm{S}_2, \mathrm{S}_3 \ldots \mathrm{S}_8$,

4. перестановка $\mathbf{P}$.

Функция $\mathbf{E}$ расширяет 32-битовый вектор R_{i-1} до 48-битового вектора $\mathbf{E}(R_{i-1})$ путём дублирования некоторых битов из R_{i-1}; порядок битов вектора $\mathbf{E}(R_{i-1})$ указан в таблице 2.

Таблица 2. Функция расширения E

32	1	2	3	4	5
4	5	6	7	8	9
8	9	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	30	31	32	1

Первые три бита вектора $\mathbf{E}(R_{i-1})$ являются битами 32, 1, 2 вектора R_{i-1}. По таблице 2 видно, что биты 1, 4, 5, 8, 9, 12, 13, 16, 17, 20, 21, 24, 25, 28, 29, 32 дублируются. Последние 3 бита вектора $E(R_{i-1})$ — это биты 31, 32, 1 вектора R_{i-1}. Полученный после перестановки блок $\mathbf{E}(R_{i-1})$ складывается по модулю 2 с ключами k_i и затем представляется в виде восьми последовательных блоков $B_1, B_2, ...B_8$.

$$\mathbf{E}(R_{i-1}) \oplus k_i = B_1B_2...B_8$$

Каждый B_j является 6-битовым блоком. Далее каждый из блоков B_j трансформируется в 4-битовый блок B'_j с помощью преобразований S_j. Преобразования S_j определяются таблицей 3.

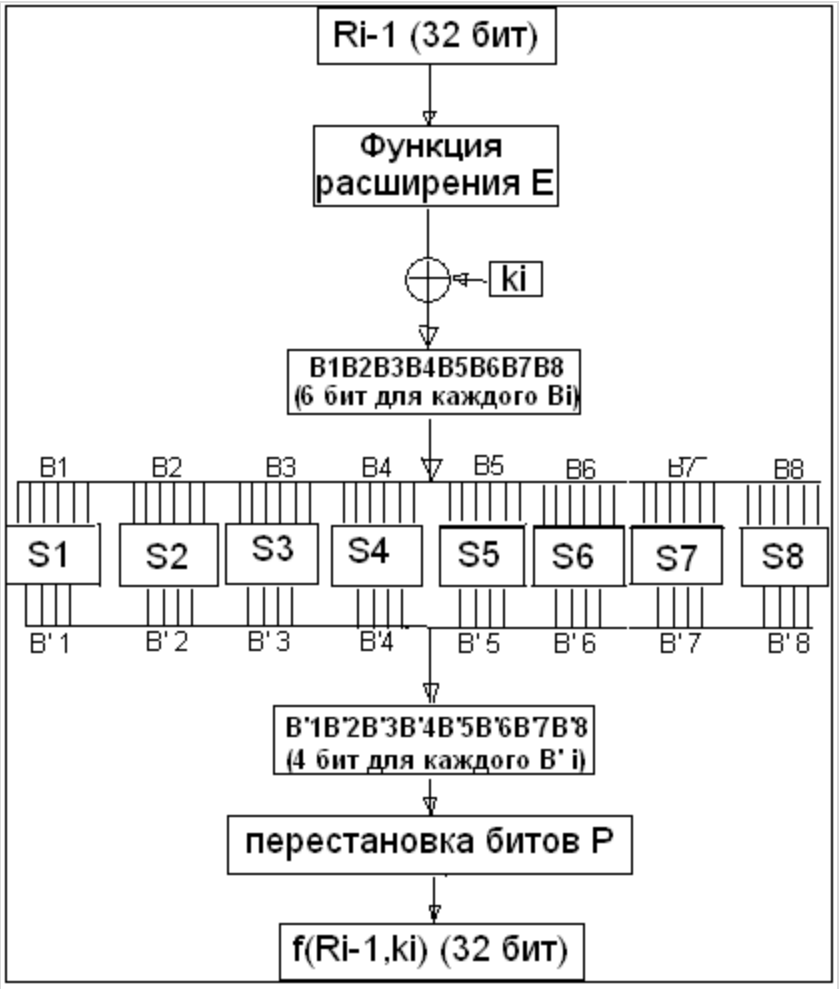


Рис.5 Схема работы функции f

Таблица 3. Преобразования S_i, i=1…8

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	
0	14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7	
1	0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8	S_1
2	4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0	
3	15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13	
0	15	1	8	14	6	11	3	4	9	7	2	13	12	0	5	10	
1	3	13	4	7	15	2	8	14	12	0	1	10	6	9	11	5	S_2
2	0	14	7	11	10	4	13	1	5	8	12	6	9	3	2	15	
3	13	8	10	1	3	15	4	2	11	6	7	12	0	5	14	9	
0	10	0	9	14	6	3	15	5	1	13	12	7	11	4	2	8	
1	13	7	0	9	3	4	6	10	2	8	5	14	12	11	15	1	S_3
2	13	6	4	9	8	15	3	0	11	1	2	12	5	10	14	7	
3	1	10	13	0	6	9	8	7	4	15	14	3	11	5	2	12	
0	7	13	14	3	0	6	9	10	1	2	8	5	11	12	4	15	
1	13	8	11	5	6	15	0	3	4	7	2	12	1	10	14	9	S_4
2	10	6	9	0	12	11	7	13	15	1	3	14	5	2	8	4	
3	3	15	0	6	10	1	13	8	9	4	5	11	12	7	2	14	
0	2	12	4	1	7	10	11	6	8	5	3	15	13	0	14	9	
1	14	11	2	12	4	7	13	1	5	0	15	10	3	9	8	6	S_5
2	4	2	1	11	10	13	7	8	15	9	12	5	6	3	0	14	
3	11	8	12	7	1	14	2	13	6	15	0	9	10	4	5	3	
0	12	1	10	15	9	2	6	8	0	13	3	4	14	7	5	11	
1	10	15	4	2	7	12	9	5	6	1	13	14	0	11	3	8	S_6
2	9	14	15	5	2	8	12	3	7	0	4	10	1	13	11	6	
3	4	3	2	12	9	5	15	10	11	14	1	7	6	0	8	13	
0	4	11	2	14	15	0	8	13	3	12	9	7	5	10	6	1	
1	13	0	11	7	4	9	1	10	14	3	5	12	2	15	8	6	S_7
2	1	4	11	13	12	3	7	14	10	15	6	8	0	5	9	2	
3	6	11	13	8	1	4	10	7	9	5	0	15	14	2	3	12	
0	13	2	8	4	6	15	11	1	10	9	3	14	5	0	12	7	
1	1	15	13	8	10	3	7	4	12	5	6	11	0	14	9	2	S_8
2	7	11	4	1	9	12	14	2	0	6	10	13	15	3	5	8	
3	2	1	14	7	4	10	8	13	15	12	9	0	3	5	6	11	

Предположим, что $B_3 = 101111$, и мы хотим найти B'_3. Первый и последний разряды B_3 являются двоичной записью числа a, 0<=a<=3, средние 4 разряда представляют число b, 0<=b<=15. Строки таблицы S3 нумеруются от 0 до 3, столбцы таблицы S3 нумеруются от 0 до 15. Пара чисел (a, b) определяет число, находящееся в пересечении строки a и столбца b. Двоичное представление этого числа дает B'_3. В нашем случае $a = 11_2 = 3$, $b = 0111_2 = 7$, а число, определяемое парой (3,7), равно 7. Его двоичное представление B'_3=0111. Значение функции $f(R_{i-1}, k_i)$ (32 бит) получается перестановкой P, применяемой к 32-битовому блоку $B'_1B'_2...B'_8$. Перестановка P задана таблицей 4.

Таблица 4. Перестановка P

16	7	20	21	29	12	28	17
1	15	23	26	5	18	31	10
2	8	24	14	32	27	3	9
19	13	30	6	22	11	4	25

$f(R_{i-1}, k_i) = P(B'_1B'_2...B'_8)$

Согласно таблице 4, первые четыре бита результирующего вектора после действия функции f — это биты 16, 7, 20, 21 вектора $B'_1B'_2...B'_8$

Генерирование ключей k_i

Ключи k_i получаются из начального ключа k (56 бит = 7 байтов или 7 символов в ASCII) следующим образом. Добавляются биты в позиции 8, 16, 24, 32, 40, 48, 56, 64 ключа k таким образом, чтобы каждый байт содержал нечетное число единиц. Это используется для обнаружения ошибок при обмене и хранении ключей. Затем делают перестановку для расширенного ключа (кроме добавляемых битов 8, 16, 24, 32, 40, 48, 56, 64). Такая перестановка определена в таблице 5.

Таблица 5.

57	49	41	33	25	17	9	1	58	50	42	34	26	18	C_0
10	2	59	51	43	35	27	19	11	3	60	52	44	36	
63	55	47	39	31	23	15	7	62	54	46	38	30	22	D_0
14	6	61	53	45	37	29	21	13	5	28	20	12	4	

Эта перестановка определяется двумя блоками C_0 и D_0 по 28 бит каждый. Первые 3 бита C_0 есть биты 57, 49, 41 расширенного ключа. А первые три бита D_0 есть биты 63, 55, 47 расширенного ключа. C_i, D_i i=1,2,3...получаются из C_{i-1}, D_{i-1} одним или двумя левыми циклическими сдвигами согласно таблице 6.

Таблица 6.

i	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Число сдвига	1	1	2	2	2	2	2	2	1	2	2	2	2	2	2	1

Ключ k_i, i=1,...16 состоит из 48 бит, выбранных из битов вектора C_iD_i (56 бит) согласно таблице 7. Первый и второй биты k_i есть биты 14, 17 вектора C_iD_i

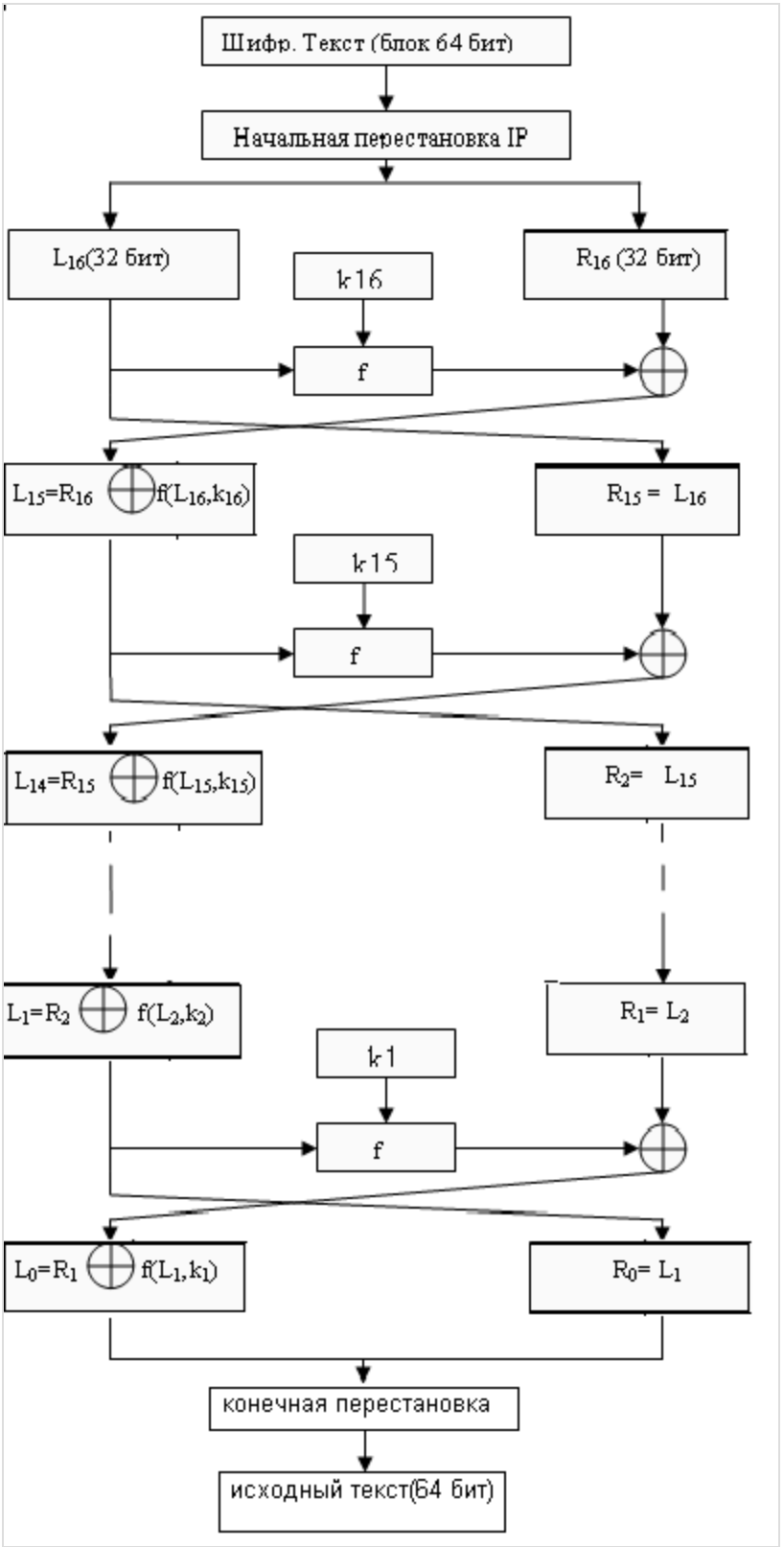


Рис.6 Схема расшифрования алгоритма DES

Таблица 7.

14	17	11	24	1	5	3	28	15	6	21	10	23	19	12	4
26	8	16	7	27	20	13	2	41	52	31	37	47	55	30	40
51	45	33	48	44	49	39	56	34	53	46	42	50	36	29	32

Конечная перестановка

Конечная перестановка IP^{-1} действует на T_{16}^{-1} (где $T_{16}^{-1} = R_{16} + L_{16}$) и является обратной к первоначальной перестановке. Конечная перестановка определяется таблицей 8.

Таблица 8. Обратная перестановка IP^{-1}

40	8	48	16	56	24	64	32	39	7	47	15	55	23	63	31
38	6	46	14	54	22	62	30	37	5	45	13	53	21	61	29
36	4	44	12	52	20	60	28	35	3	43	11	51	19	59	27
34	2	42	10	50	18	58	26	33	1	41	9	49	17	57	25

Схема расшифрования

При расшифровании данных все действия выполняются в обратном порядке. В 16 циклах расшифрования, в отличие от шифрования с помощью прямого преобразования сетью Фейстеля, здесь используется обратное преобразование сетью Фейстеля.

$$R_{i-1} = L_i$$
$$L_{i-1} = R_i \oplus f(L_i, k_i)$$

Схема расшифрования указана на Рис.6.
Ключ k_i, i=16,...,1, функция f, перестановка IP и IP^{-1} такие же, как и в процессе шифрования. Алгоритм генерации ключей зависит только от ключа пользователя, поэтому при расшифровании они идентичны.

Псевдокод

```
// Все переменные беззнаковые 64-битные

// Предварительная обработка: дополнение разницы в размере в байтах
Дополнить сообщение до кратного 64 битам

var key // Ключи, предоставленные пользователем
var keys[16]
var left, right

// Генерация ключей

// PC1 (64 бита в 56 бит)
key := перестановка(key, PC1)
left := (key сдвиг вправо на 28 бит) and 0xFFFFFFF
right := key и 0xFFFFFFF

for i от 0 до 16 do
    right := right циклический сдвиг влево на KEY_shift[i]
    left := left циклический сдвиг влево на KEY_shift[i]
    var concat := (left сдвиг влево на 28 бит) or right
    // PC2 (56 бит в 48 бит)
    keys[i] := перестановка(concat, PC2)
end for

// Для расшифрования сообщения изменить порядок ключей
if decrypt do
    reverse keys
end if

// Шифрование или расшифрование
```

```
for каждый 64-битный блок дополненного сообщения do
    var tmp

    // IP
    chunk := перестановка(chunk, IP)
    left := chunk сдвиг вправо на 32 бита
    right := chunk and 0xFFFFFFFF
    for i от 0 до 16 do
        tmp := right
        // E (32 бита в 48 бит)
        right := расширение(right, E)
        right := right xor keys[i]
        // Подстановка (48 бит в 32 бита)
        right := подстановка(right)
        // P
        right := перестановка(right, P)
        right := right xor left
        left := tmp
    end for
    // Объединить right и left
    var cipher_chunk := (right сдвиг влево на 32 бита) or left
    // Конечная перестановка
    cipher_chunk := перестановка(cipher_chunk, FP)
end for
```

Режимы использования DES

DES может использоваться в четырёх режимах.

1. Режим электронной кодовой книги (ECB — Electronic Codebook): обычное использование DES как блочного шифра. Шифруемый текст разбивается на блоки, при этом каждый блок шифруется отдельно, не взаимодействуя с другими блоками (см. Рис.7).

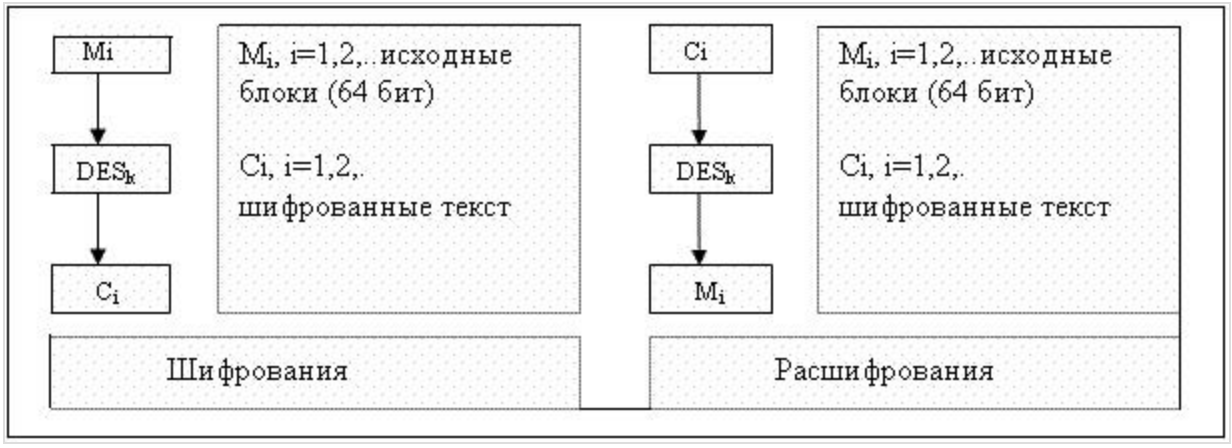


Рис.7 Режим электронной кодовой книги — ECB

2. Режим сцепления блоков шифротекста (Cipher Block Chaining) (см. Рис.8). Каждый очередной блок M_i i>=1, перед зашифровыванием складывается по модулю 2 с предыдущим блоком зашифрованного текста C_{i-1}. Вектор C_0 — начальный вектор, он меняется ежедневно и хранится в секрете.

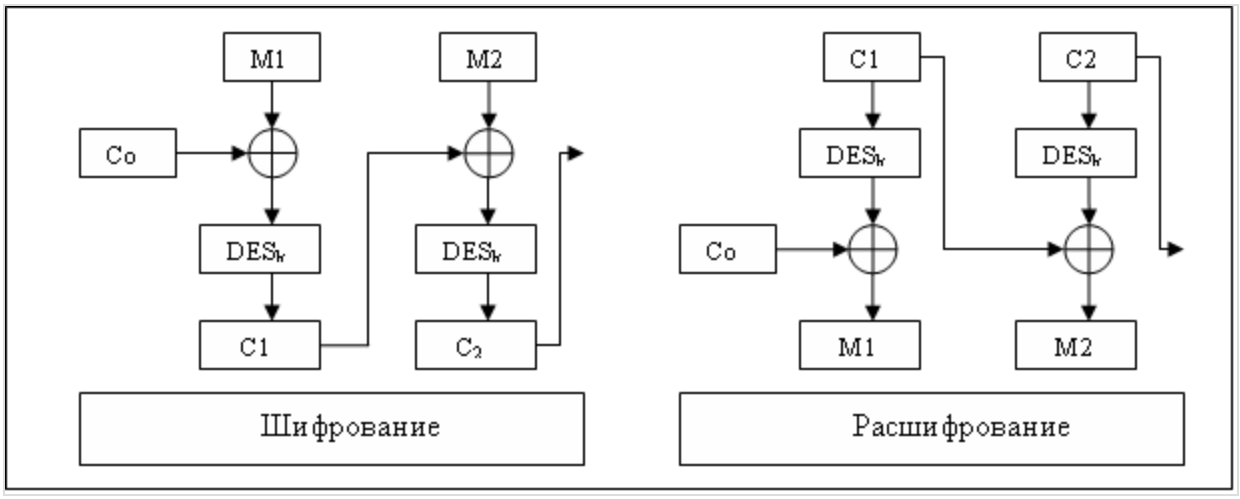


Рис.8 Режим сцепления блоков — CBC

3. Режим обратной связи по шифротексту (Cipher Feedback) (см. Рис.9). В режиме CFB вырабатывается блочная «гамма» $Z_0, Z_1, ...Z_i = DES_k(C_{i-1})\ C_i = M_i \oplus Z_i$. Начальный вектор C_0 является синхропосылкой и предназначен для того, чтобы разные наборы данных шифровались по-разному с использованием одного и того же секретного ключа. Синхропосылка посылается получателю в открытом виде вместе с зашифрованным файлом. Алгоритм DES, в отличие от предыдущих режимов, используется только как шифрование (в обоих случаях).

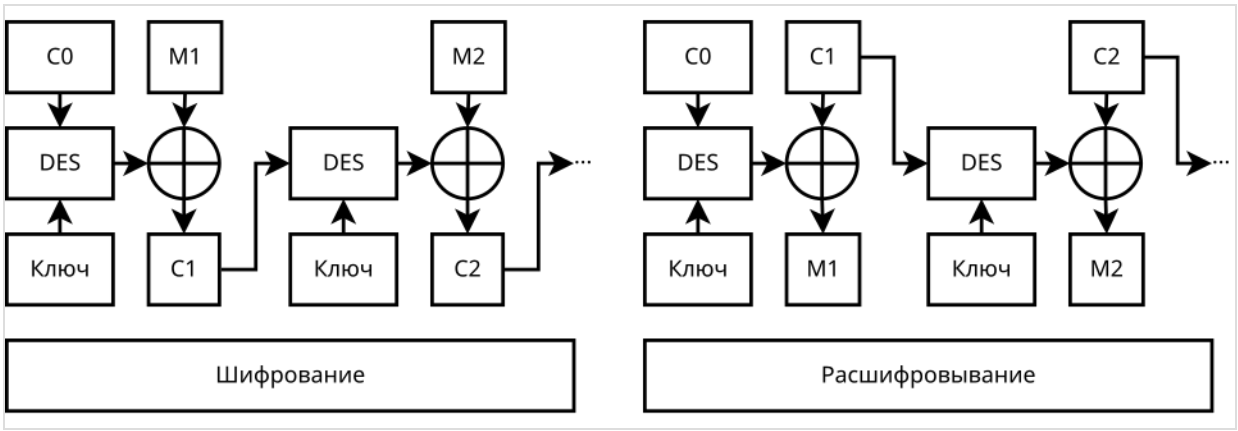


Рис.9 Режим обратной связи по шифротексту — CFB

4. Режим обратной связи по выходу (OFB — Output Feedback) (см. Рис.10). В режиме OFB вырабатывается блочная «гамма» $Z_0, Z_1, ... Z_i = DES_k(Z_{i-1}) C_i = M_i \oplus Z_i$, i>=1. Режим также использует DES только как шифрование (в обоих случаях).

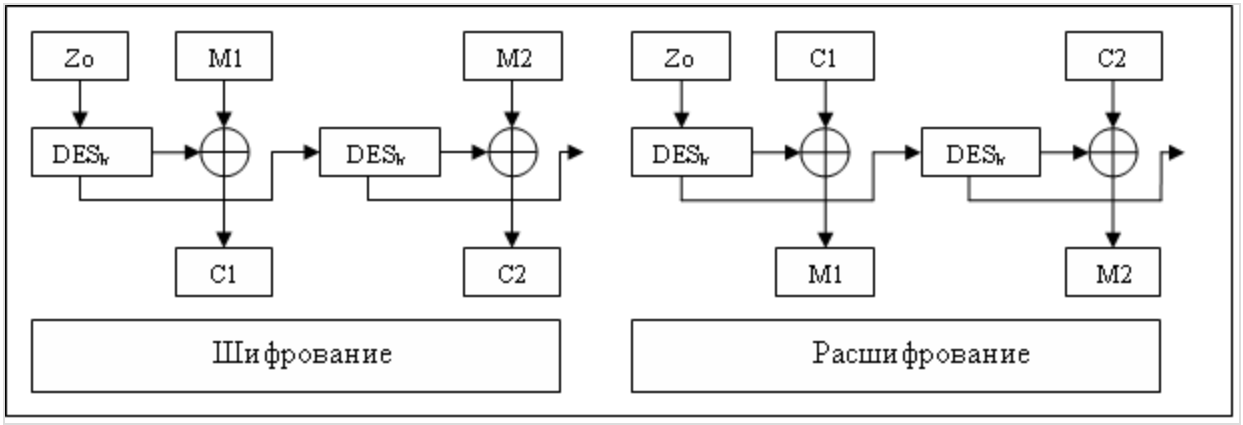


Рис.10 Режим обратной связи по выходу — OFB

Достоинства и недостатки режимов

В режимах ECB и OFB искажение при передаче одного 64-битового блока шифротекста C_i приводит к искажению после расшифрования только соответствующего открытого блока M_i, поэтому такие режимы используется для передачи по каналам связи с большим числом искажений.

Криптостойкость алгоритма DES

Нелинейность преобразований в DES средствами только S-блоков и использование слабых S-блоков позволяет осуществлять контроль над шифрованной перепиской. Выбор S-блоков требует соблюдения нескольких условий:

- Каждая строка каждого блока должна быть перестановкой множества {0, 1, 2, …, 15}
- S-блоки не должны являться линейной или афинной функцией своих аргументов.
- Изменение одного бита на входе S-блока должно приводить к изменению, по крайней мере, двух битов на выходе.
- Для каждого S-блока и любого аргумента *x* значение *S*(*x*) и $S(x \oplus 001100_2)$ должны различаться, по крайней мере, двумя битами.

Из-за небольшого числа возможных ключей (всего 2^{56}), появляется возможность их полного перебора на быстродействующей вычислительной технике за реальное время. В 1998 году Electronic Frontier Foundation, используя специальный компьютер DES-Cracker, удалось взломать DES за 3 дня.

Слабые ключи

Слабыми ключами называется ключи k такие, что $DES_k(DES_k(x)) = x$, где x — 64-битный блок.

Известны 4 слабых ключа, они приведены в таблице 9. Для каждого слабого ключа существует 2^{32} неподвижные точки, то есть таких 64-битных блоков x, для которых $DES_k(x) = x$.

Таблица 9. DES-Слабые ключи

Слабые ключи (hexadecimal)	C_0	D_0
0101-0101-0101-0101	$[0]^{28}$	$[0]^{28}$
FEFE-FEFE-FEFE-FEFE	$[1]^{28}$	$[1]^{28}$
1F1F-1F1F-0E0E-0E0E	$[0]^{28}$	$[1]^{28}$
E0E0-E0E0-F1F1-F1F1	$[1]^{28}$	$[0]^{28}$

$[0]^{28}$ обозначает вектор, состоящий из 28 нулевых битов.

Частично слабые ключи

В алгоритме DES существуют слабые и частично слабые ключи. Частично слабые ключи — это такие пары ключей (k_1, k_2), что $DES_{k1}(DES_{k2}(x)) = x$.

Существуют 6 частично слабых пар ключей, они приведены в таблице 10. Для каждого из 12 частично слабых ключей существуют 2^{32} «антинеподвижные точки», то есть такие блоки x, что $DES_k(x) = \tilde{x}$.

Таблица 10. Частично слабые ключи

C_0	D_0	Пары частично слабых ключей	C_0	D_0
$[01]^{14}$	$[01]^{14}$	01FE-01FE-01FE-01FE,----FE01-FE01-FE01-FE01	$[10]^{14}$	$[10]^{14}$
$[01]^{14}$	$[01]^{14}$	1FE0-1FE0-1FE0-1FE0,----E0F1-E0F1-E0F1-E0F1	$[10]^{14}$	$[10]^{14}$
$[01]^{14}$	$[0]^{28}$	01E0-01E0-01F1-01F1,----E001-E001-F101-F101	$[10]^{14}$	$[0]^{28}$
$[01]^{14}$	$[1]^{28}$	1FFE-1FFE-0EFE-0EFE,----FE1F-FE1F-FE0E-FE0E	$[0]^{28}$	$[1]^{28}$
$[0]^{28}$	$[01]^{14}$	011F-011F-010E-010E,----1F01-1F01-0E01-0E01	$[0]^{28}$	$[10]^{14}$
$[1]^{28}$	$[01]^{14}$	E0FE-E0FE-F1FE-F1FE,----FEE0-FEE0-FEF1-FEF1	$[1]^{28}$	$[10]^{14}$

Известные атаки на DES

Таблица 11. Известные атаки на DES.

Методы атаки	Известные откр. тексты	Выбранные отк. тексты	Объём памяти	Количество операций
Полный поиск	qweqweqweqerqe	-	Незначительный	2^{55}
Линейный Криптоанализ	$2^{43}(85\%)$	-	Для текста	2^{43}
Линейный Криптоанализ	$2^{38}(10\%)$	-	Для текста	2^{50}
Диффер. Криптоанализ	-	2^{47}	Для текста	2^{47}
Диффер. Криптоанализ	2^{55}	-	Для текста	2^{55}

- Метод полного перебора требует одну известную пару шифрованного и расшифрованного текста, незначительный объём памяти, и его выполнение требует около 2^{55} шагов.
- Дифференциальный криптоанализ — первую такую атаку на DES заявили Бихам и Шамир. Эта атака требует шифрования 2^{47} открытых текстов, выбранных нападающим, и для её выполнения нужны примерно 2^{47} шагов. Теоретически являясь точкой разрыва, эта атака непрактична из-за чрезмерных требований к подбору данных и сложности организации атаки по выбранному открытому тексту. Сами авторы этой атаки Biham и Shamir заявили, что считают DES защищённым для такой атаки.
- Линейный криптоанализ разработан Matsui. Этот метод позволяет восстановить ключ DES с помощью анализа 2^{43} известных открытых текстов, при этом требуется примерно 2^{43} шагов для выполнения. Первый экспериментальный криптоанализ DES, основанный на открытии

Matsui, был успешно выполнен в течение 50 дней на автоматизированных рабочих местах 12 HP 9735.

Для линейного и дифференциального криптоанализа требуется достаточно большой объём памяти для сохранения выбранных (известных) открытых текстов до начала атаки.

Увеличение криптостойкости DES

Чтобы увеличивать криптостойкость DES, появляются несколько вариантов: double DES (2DES), triple DES (3DES), DESX, G-DES.

- Методы 2DES и 3DES основаны на DES, но увеличивают длину ключей (2DES — 112 бит, 3DES — 168 бит) и поэтому увеличивается криптостойкость.
- Схема 3DES имеет вид $DES(k_3, DES(k_2, DES(k_1, M)))$, где k_1, k_2, k_3 ключи для каждого шифра DES. Это вариант известен как в EEE, так как три DES операции являются шифрованием. Существует 3 типа алгоритма 3DES:
  - DES-EEE3: Шифруется три раза с 3 разными ключами.
  - DES-EDE3: 3DES операции шифровка-расшифровка-шифровка с 3 разными ключами.
  - DES-EEE2 и DES-EDE2: Как и предыдущие, за исключением того, что первая и третья операции используют одинаковый ключ.
- Самый популярный тип при использовании 3DES — это DES-EDE3, для него алгоритм выглядит так:
  - Зашифрование: $C = E_{k_3}(E_{k_2}^{-1}(E_{k_1}(P)))$.
  - Расшифрование: $P = E_{k_1}^{-1}(E_{k_2}(E_{k_3}^{-1}(C)))$
- При выполнении алгоритма 3DES ключи могут выбрать так:
  - k_1, k_2, k_3 независимы.
  - k_1, k_2 независимы, а $k_1 = k_3$
  - $k_1 = k_2 = k_3$.
- Метод DESX создан Рональдом Ривестом и формально продемонстрирована Killian и Rogaway. Этот метод — усиленный вариант DES, поддерживаемый инструментарием RSA Security. DESX отличается от DES тем, что каждый бит входного открытого текста DESX логически суммируется по модулю 2 с 64 битами дополнительного ключа, а затем шифруется по алгоритму DES. Каждый бит результата также логически суммируется по модулю 2 с другими 64 битами ключа. Главной причиной использования DESX является простой в вычислительном смысле способ значительно повысить стойкость DES к атакам полного перебора ключа.
- Метод G-DES разработан Schaumuller-Bichl для повышения производительности DES на основе увеличения размеров шифрованного блока. Заявлялось, что G-DES защищён так же, как и DES. Однако Biham и Shamir показали, что G-DES с рекомендуемыми параметрами легко взламывается, а при любых изменениях параметров шифр становится ещё менее защищён, чем DES.
- Ещё другой вариант DES использует независимые суб-ключи. Различно от алгоритма DES, в котором на основе 56-битного секретного ключа пользователя алгоритм DES получает шестнадцать 48-битных суб-ключей для использования в каждом из 16 раундов, в этом варианте использует 768-битный ключ (разделённый на 16 48-битных подключей) вместо 16 зависимых 48-битных ключей, создаваемых по ключевому графику алгоритма DES. Хотя очевидно, что использование независимых суб-ключей значительно усложнит полный поиск

ключа, но стойкость к атаке дифференциальным или линейным криптоанализом ненамного превысит стойкость обычного DES. По оценке Biham для дифференциального криптоанализа DES с независимыми подключами требуется 2^{61} выбранных открытых текстов, в то время как для линейного криптоанализа требуется 2^{60} известных открытых текстов.

Применение

DES был национальным стандартом США в 1977—1980 гг., но в настоящее время DES используется (с ключом длины 56 бит) только для устаревших систем, чаще всего используют его более криптоустойчивый вид (3DES, DESX). 3DES является простой эффективной заменой DES, и сейчас он рассмотрен как стандарт. В ближайшее время DES и Triple DES будут заменены алгоритмом AES (Advanced Encryption Standard — Расширенный Стандарт Шифрования).

Алгоритм DES широко применяется для защиты финансовой информации: так, модуль Thales (Racal) HSM RG7000 полностью поддерживает операции TripleDES для эмиссии и обработки кредитных карт VISA, EuroPay и проч. Канальные шифраторы THALES (Racal) DataDryptor 2000 используют TripleDES для прозрачного шифрования потоков информации. Также алгоритм DES используется во многих других устройствах и решениях THALES-eSECURITY.

Примечания

1. distributed.net: Project DES (https://www.distributed.net/DES) (англ.). Дата обращения: 12 декабря 2025. Архивировано (https://web.archive.org/web/20171231111343/http://www.distributed.net/DES) 31 декабря 2017 года.

Литература

- *А. П. Алферов, А. Ю. Зубов, А. С. Кузьмин, А. В. Черемушкин* . Основы криптографии.
- *A. Menezes, Pvan Oorschot, S. Vanstone.* Handbook of Applied Cryptography (https://cacr.uwaterloo.ca/hac/) (англ.).
- *Семенов Ю. А.* Алгоритм DES (http://book.itep.ru/6/des_641.htm).
- Query for DES (http://everything2.com/title/DES)

Эта статья **нуждается в переработке**. **Пожалуйста, уточните проблему в статье с помощью более конкретного шаблона.**
Пожалуйста, улучшите статью в соответствии с правилами написания статей. *(16 декабря 2008)*

Источник — https://ru.wikipedia.org/w/index.php?title=DES&oldid=150601395